Aim

To design and train a **Convolutional Neural Network (CNN)** using **TensorFlow/Keras** on an image dataset (MNIST/CIFAR-10) and evaluate its performance for image classification.

Theory

A **Convolutional Neural Network (CNN)** is a deep learning model specifically designed for **image processing and computer vision tasks**.

Key Layers in CNN:

1. **Convolution Layer**
    - Applies filters/kernels to extract features (edges, textures)
    - Produces feature maps
2. **Activation Function**
    - Common: ReLU (Rectified Linear Unit)
3. **Pooling Layer**
    - Reduces spatial size (MaxPooling)
    - Helps reduce overfitting
4. **Flatten Layer**
    - Converts 2D feature maps into 1D vector
5. **Fully Connected Layer**
    - Performs classification

Advantages of CNN:

- Automatically extracts features
- Reduces manual feature engineering
- High accuracy in image classification

Procedure

1. Import required libraries
2. Load dataset (MNIST/CIFAR-10)
3. Normalize image data
4. Reshape input for CNN
5. Build CNN model
6. Compile the model

7. Train the model
8. Evaluate accuracy
9. Predict outputs

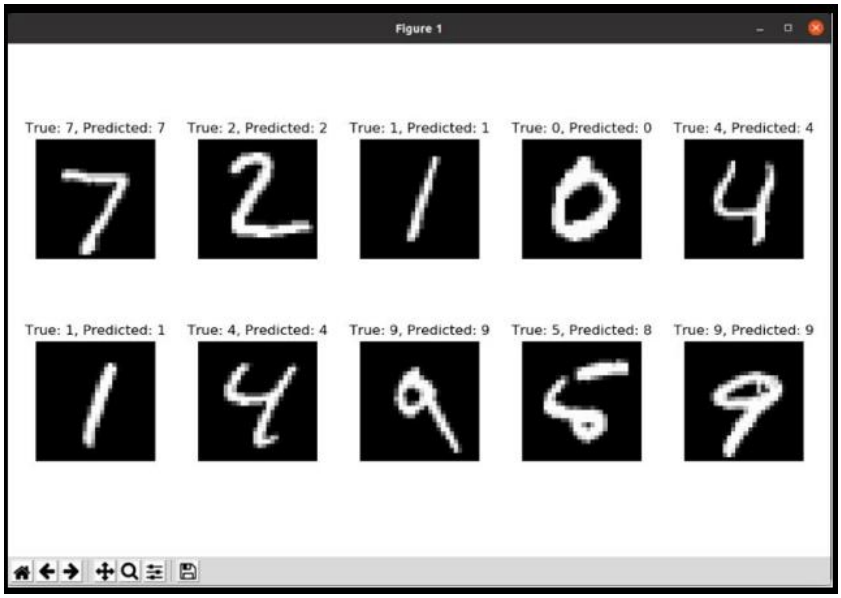
CODE:

```
# Step 1: Import Libraries
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt

# Step 2: Load Dataset
(X_train, y_train), (X_test, y_test) = datasets.mnist.load_data()

# Step 3: Normalize Data
X_train = X_train / 255.0
X_test = X_test / 255.0

# Step 4: Reshape (for CNN)
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# Step 5: Build CNN Model
model = models.Sequential()

model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(layers.MaxPooling2D((2,2)))

model.add(layers.Conv2D(64, (3,3), activation='relu'))
model.add(layers.MaxPooling2D((2,2)))

model.add(layers.Flatten())
```

```
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

# Step 6: Compile Model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Step 7: Train Model
model.fit(X_train, y_train, epochs=5)

# Step 8: Evaluate Model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)

# Step 9: Prediction
predictions = model.predict(X_test)
print(predictions[0])
# Display first 10 images with predictions
for i in range(10):
    plt.imshow(X_test[i].reshape(28, 28), cmap='gray')

    # Get predicted label
    predicted_label = np.argmax(predictions[i])

    # Actual label
    actual_label = y_test[i]

    plt.title(f"Predicted: {predicted_label}, Actual: {actual_label}")
    plt.axis('off')
    plt.show()
```

OUTPUT:

```
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 246s 123ms/step - accuracy: 0.9860 - loss: 0.0463
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 252s 117ms/step - accuracy: 0.9897 - loss: 0.0323
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 215s 115ms/step - accuracy: 0.9924 - loss: 0.0233
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 256s 111ms/step - accuracy: 0.9948 - loss: 0.0175
313/313 ━━━━━━━━━━━━━━━━━━━━ 25s 77ms/step - accuracy: 0.9909 - loss: 0.0287
Test Accuracy: 0.9908999800682068
313/313 ━━━━━━━━━━━━━━━━━━━━ 12s 38ms/step
[1.3709154e-09 1.1141572e-07 1.3323873e-06 3.3004842e-05 2.0284526e-07
 3.5322092e-09 2.8798436e-14 9.9996519e-01 3.4373797e-09 1.7408652e-07]
```



Conclusion:

A Convolutional Neural Network (CNN) was successfully designed and trained using TensorFlow/Keras on the MNIST dataset. The model effectively learned image features such as edges and shapes, achieving high classification accuracy. This experiment demonstrates the power of CNNs in image processing tasks and their importance in real-world applications like handwriting recognition, facial recognition, and object detection